<table>
<tr><td colspan="4">Name of Students: Om Wadhwani</td></tr>
<tr><td colspan="2">Roll Number: 58</td><td colspan="2">Lab Assignment Number: 10</td></tr>
<tr><td colspan="4">Title of Lab Assignment: Assignment based on Spring boot</td></tr>
<tr><td colspan="2">DOP: 24/11/25</td><td colspan="2">DOS: 08/12/25</td></tr>
<tr><td>CO Mapped:
CO5,CO6</td><td>PO Mapped:
PO1,PO2,PO4,PSO1,
PSO2</td><td>Faculty Signature:</td><td>Marks:</td></tr>
</table>

Number Format Exception Roll No, Name, Marks, Subject

A *NumberFormatException* in Java occurs when a program attempts to convert a string into a numeric value, but the string does not contain a valid numeric representation. This exception belongs to the java.lang package and is a type of unchecked exception. In this program, the user enters **Roll Number, Name, Marks, and Subject** using a command-line interface or HTML form. The Roll Number and Marks must be numeric values, while Name and Subject are strings. When the program tries to convert Roll Number or Marks using functions like Integer.parseInt() or Double.parseDouble(), any invalid numeric input (such as characters, spaces, or symbols) will immediately cause a *NumberFormatException*. This exception ensures that incorrect or non-numeric input does not break the logic of the program. The program catches this exception and displays a proper error message to the user, ensuring smooth execution. Finally, when the correct numeric input is provided, the program calculates the marks percentage and displays the student's details.

User-Defined Exception in Bank Program

In Java, a *user-defined exception* is a custom exception created by the programmer to handle specific application errors that are not covered by built-in exceptions. In the Bank application, the program enforces a **minimum balance rule of ₹500**, meaning the user's account balance must never fall below this limit. To handle violations of this rule, a custom exception such as PayoutOutOfBoundsException is created. This exception is thrown whenever the user attempts to open an account with less than ₹500, withdraw an amount that reduces the balance below ₹500, or enter an invalid amount. The Bank class contains the core banking operations—**open**, **deposit**, and **withdraw**—and each method checks whether the transaction obeys the minimum balance rule. If not, the custom exception is fired, and the program displays a meaningful error message instead of terminating abnormally. This approach increases program reliability, provides better error handling, and allows customized business logic in financial applications. Using a custom exception also demonstrates how Java supports object-oriented programming through exception handling and class design.

Q1Number Format Exception Roll No, Name, Marks, Subject

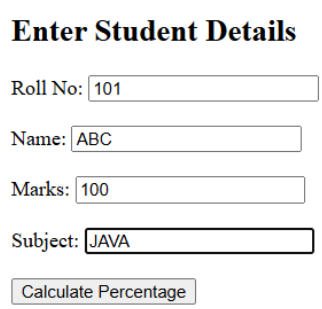
StudentController.java

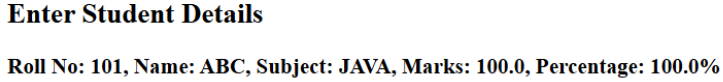
```
package com.practical8APractice.practical8APractice;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
@Controller
@RequestMapping("/student")
public class StudentController {
  @GetMapping("/")
  public String showForm() {
     return "studentform";  // loads HTML form
  }
  @PostMapping("/calculate")
  public String calculatePercentage(
        @RequestParam String roll,
        @RequestParam String name,
        @RequestParam String marks,
        @RequestParam String subject,
        Model model) {
     try {

        int rollNo = Integer.parseInt(roll);
        double markValue = Double.parseDouble(marks);
        double percentage = (markValue / 100) * 100;
        model.addAttribute("message",
              "Roll No: " + rollNo +
              ", Name: " + name +
              ", Subject: " + subject +
              ", Marks: " + markValue +
              ", Percentage: " + percentage + "%");
     } catch (NumberFormatException e) {
        model.addAttribute("message",
              " Number Format Error: Please enter numbers only for Roll No and Marks!");
     }
     return "studentform"; // Reload form with result/error
  }
}
```

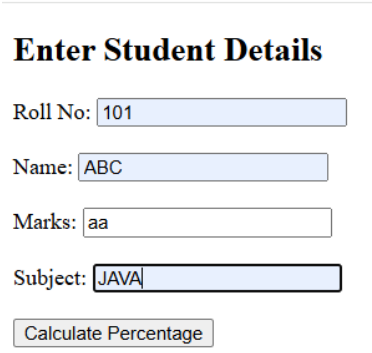
StudentForm.html

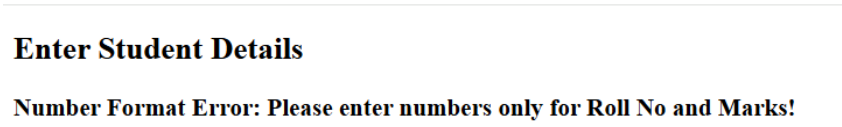
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Student Percentage Calculator</title>
</head>
<body>
<h2>Enter Student Details</h2>
<!-- Show error or result -->
<h3 th:text="${message}"></h3>
<form action="/student/calculate" method="post">
  <label>Roll No:</label>
  <input type="text" name="roll" required><br><br>
  <label>Name:</label>
  <input type="text" name="name" required><br><br>
  <label>Marks:</label>
  <input type="text" name="marks" required><br><br>
  <label>Subject:</label>
  <input type="text" name="subject" required><br><br>
  <button type="submit">Calculate Percentage</button>
</form>
</body>
</html>
```

Output:









Q2User-Defined Exception in Bank Program

Bank.java

```
package com.practical8APractice.practical8APractice;
import org.springframework.stereotype.Service;
@Service
public class Bank {
  private double balance;
  public void open(double openingAmount) {
     if (openingAmount < 500) {
        throw new PayoutOutOfBoundsException("Opening balance must be at least 500!");
     }
     this.balance = openingAmount;
  }
  public void deposit(double amount) {
     if (amount <= 0) {
        throw new PayoutOutOfBoundsException("Deposit amount must be positive!");
     }
     balance += amount;
  }
  public void withdraw(double amount) {
     if (amount <= 0) {
        throw new PayoutOutOfBoundsException("Withdrawal amount must be positive!");
     }
     if ((balance - amount) < 500) {
        throw new PayoutOutOfBoundsException("Withdrawal denied! Minimum balance of
500 must remain.");
     }
     balance -= amount;
```

```
  }
  public double getBalance() {
    return balance;
  }
}
```

BankApplication.java

```
package com.practical8APractice.practical8APractice;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class BankApplication {
  public static void main(String[] args) {
    SpringApplication.run(BankApplication.class, args);
  }
}
```

BankController.java

```
package com.practical8APractice.practical8APractice;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
@Controller
@RequestMapping("/bank")
public class BankController {
  @Autowired
  private Bank bank;
  @GetMapping("/")
  public String showForm() {
    return "bankform"; // loads bankform.html
  }
  @PostMapping("/open")
  public String openAccount(@RequestParam double amount, Model model) {
    bank.open(amount);
    model.addAttribute("message", "Account opened. Current Balance: " + bank.getBalance());
    return "bankform";
```

```
  }
  @PostMapping("/deposit")
  public String deposit(@RequestParam double amount, Model model) {
     bank.deposit(amount);
     model.addAttribute("message", "Deposit successful. New Balance: " + bank.getBalance());
     return "bankform";
  }
  @PostMapping("/withdraw")
  public String withdraw(@RequestParam double amount, Model model) {
     bank.withdraw(amount);
     model.addAttribute("message", "Withdraw successful. New Balance: " +
bank.getBalance());
     return "bankform";
  }
}
```

PayoutOutOfBoundsException.java

```
package com.practical8APractice.practical8APractice;
public class PayoutOutOfBoundsException extends RuntimeException {
  public PayoutOutOfBoundsException(String message) {
     super(message);
  }
}
```

BankForm.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Bank Form</title>
</head>
<body>
<h2>Bank Operations</h2>
<h3 th:text="${message}"></h3>

<form action="/bank/open" method="post">
  <h4>Open Account</h4>
  <label>Opening Amount:</label>
  <input type="number" name="amount" required>
```

```
  <button type="submit">Open Account</button>
</form>
<br/>

<form action="/bank/deposit" method="post">
  <h4>Deposit Money</h4>
  <label>Deposit Amount:</label>
  <input type="number" name="amount" required>
  <button type="submit">Deposit</button>
</form>
<br/>

<form action="/bank/withdraw" method="post">
  <h4>Withdraw Money</h4>
  <label>Withdraw Amount:</label>
  <input type="number" name="amount" required>
  <button type="submit">Withdraw</button>
</form>
</body>
</html>
```

Output

Bank Operations

Account opened. Current Balance: 1000.0

Bank Operations

Account opened. Current Balance: 1000.0

Open Account

Opening Amount: [ ] Open Account

Deposit Money

Deposit Amount: 100 Deposit

Withdraw Money

Withdraw Amount: [ ] Withdraw

Bank Operations

Deposit successful. New Balance: 1100.0

Bank Operations

Deposit successful. New Balance: 1100.0

Open Account

Opening Amount: [ ] Open Account

Deposit Money

Deposit Amount: [ ] Deposit

Withdraw Money

Withdraw Amount: 500 Withdraw

Bank Operations

Withdraw successful. New Balance: 600.0

Open Account

Opening Amount: [ ] Open Account

Deposit Money

Deposit Amount: [ ] Deposit

Withdraw Money

Withdraw Amount: 300 Withdraw

Bank Operations

Withdraw successful. New Balance: 600.0

Error: Withdrawal denied! Minimum balance of 500 must remain.